

Sprawozdanie Mykhailo Shevchenko, ms420826 na temat zadania zaliczeniowego z C++ z Programowania Współbieżnego

Napisałem dwie wersje programu, które przetestowałem na dwóch systemach – students oraz lokalnie, na Ubuntu wersji 20. Niżej są odpowiednio charakterystyki komputerów. Sprawdzone poleceniem “lscpu”. Sprawdzenie ilości RAMu za pomocą “free -h” na students daje 328GB a lokalnie 7.6GB.

Architektura: x86_64
Tryb(y) pracy CPU: 32-bit, 64-bit
Kolejność bajtów: Little Endian
Rozmiary adresów: 40 bits physical, 48 bits virtual
CPU: 64
Lista aktywnych CPU: 0-63
Wątków na rdzeń: 1
Rdzeni na gniazdo: 1
Gniazdo: 64
Węzłów NUMA: 1
ID producenta: GenuineIntel
Rodzina CPU: 6
Model: 85
Nazwa modelu: Intel Xeon Processor (Skylake, IBRS)
Wersja: 4
CPU MHz: 2095.078
BogoMIPS: 4190.15
Producent hipernadzorcy: KVM
Typ wirtualizacji: pełna
Cache L1d: 32K
Cache L1i: 32K
Cache L2: 4096K
Cache L3: 16384K
Procesory węzła NUMA 0: 0-63

Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Byte Order: Little Endian
Address sizes: 39 bits physical, 48 bits virtual
CPU(s): 8
On-line CPU(s) list: 0-7
Thread(s) per core: 2
Core(s) per socket: 4
Socket(s): 1
NUMA node(s): 1
Vendor ID: GenuineIntel
CPU family: 6
Model: 142
Model name: Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz
Stepping: 12
CPU MHz: 800.007
CPU max MHz: 4200,0000

CPU min MHz: 400,0000
BogoMIPS: 4199.88
Virtualization: VT-x
L1d cache: 128 KiB
L1i cache: 128 KiB
L2 cache: 1 MiB
L3 cache: 6 MiB
NUMA node0 CPU(s): 0-7
Vulnerability Itlb multihit: KVM: Mitigation: Split huge pages
Vulnerability L1tf: Not affected
Vulnerability Mds: Not affected
Vulnerability Meltdown: Not affected
Vulnerability Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl and seccomp
Vulnerability Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Vulnerability Spectre v2: Mitigation; Enhanced IBRS, IBPB conditional, RSB filling
Vulnerability Srbds: Mitigation; TSX disabled
Vulnerability Tsx async abort: Not affected

Opis algorytmów

Swoje rozwiązanie współbieżne bazowałem na sekwencyjnym algorytmie. Pisanie projektu zacząłem od obserwacji, że liczenie wynikowego wektora PageRank w każdej iteracji się nie opłaca i że możemy to zrobić pod sam koniec, kiedy błąd przybliżenia jest dostatecznie niski, co nieco przyspiesza rozwiązanie sekwencyjne.

Rozwiązanie sekwencyjne składa się z następujących etapów:

- 1) generowanie id dla każdej strony
- 2) tłumaczenie klasy Network na wygodne pomocnicze struktury danych (tworzenie map, wektorów itd.)
- 3) liczenie pageRank w iteracjach

Najważniejszym zadaniem jest zrównoleglenie liczenia pageRank w iteracjach, gdyż jest intuicyjnie jasne, że ta faza algorytmu wymaga najwięcej pracy od procesora. Natomiast nie mniej ważne jest zrównoleglenie generowania id, ponieważ jest to łatwo zrównoleglić ze względu na niezależność generowania id jednej strony od drugiej, a wygenerowanie id też wymaga potencjalnie dużo czasu (uruchomienie zewnętrznego programu jakim jest sha256sum, operowanie na dużych napisach). Jeśli chodzi o drugą fazę algorytmu to jest ona stosunkowo krótka i nie wymaga zrównoleglenia. Co więcej, jej zrównoleglenie bardzo prawdopodobnie mogłoby się okazać nietrywialne. Należy zauważyć, że dla prawidłowego działania etapy algorytmu powinny zostać wykonane sekwencyjnie, a trzeci etap algorytmu powinien obliczać pageRank sekwencyjnie ze względu na iteracje.

Zatem pierwszą fazę algorytmu realizuję, dzieląc strony po równo na liczbę dostępnych wątków a drugą fazę wykonuję sekwencyjnie.

Liczenie pageRank w danej iteracji również składa się z kilku etapów:

- 1) liczenie sumy wag wiszących węzłów
- 2) wstępne ustawienie PageRank dla wszystkich stron (wkład niezależny od sąsiadów)
- 3) zaktualizowanie PageRank w zależności od PageRank sąsiadów z ubiegłej iteracji
- 4) obliczenie błędu przybliżenia
- 5) jeśli błąd jest mały to stworzenie wynikowego wektora i zakończenie algorytmu
- 6) jeśli nie to zaktualizowanie ostatniej wersji PageRank

Należy zauważyć, że 1) musi się odbyć przed 2); 2) i 3) muszą się odbyć przed 4) oraz przed 6). Piąty etap wykonujemy sekwencyjnie, ponieważ jest to jednorazowy zabieg. Każdy inny etap wymaga stosunkowo dużo pracy, więc skoro one muszą się odbywać sekwencyjnie (z przymrużeniem oka), to najpierw wszystkie wątki są przeznaczane na pierwszy etap, potem na drugi i tak dalej.

Pierwszy, drugi i czwarty etap są analogiczne pod względem zrównoleglenia, po prostu dzielimy strony na liczbę wątków po równo. Wątki nie muszą podczas wykonywania etapu się komunikować, ponieważ działają na rozłącznych zbiorach stron, i też mają bardzo podobne objętości pracy.

Trzeci etap liczenia iteracji wymaga najwięcej inwencji i pozwala na parę rozwiązań, dwa z których spróbowałem zaimplementować, z których oddałem pierwsze. Oba pomysły operują się na następującej idei. W tym etapie, w przeciwieństwie do poprzednich, dzielenie stron po równo na wątki nie rozdziela pracy równomiernie, ponieważ do jednej strony może wchodzić więcej linków niż do drugiej. Natomiast jeśli mamy zbiór krawędzi w grafie, to jego już możemy równomiernie podzielić.

Zatem w oddanym rozwiązaniu, każdy wątek ma lokalnie mapę PageRank dla stron, które są w zbiorze krawędzi, którymi ten wątek ma się zająć. Wątek dodaje PageRank dla tych krawędzi. Jak wszystkie wątki skończą to robić, wątek główny synchronizuje i kombinuje wartości z lokalnych map.

W rozwiązaniu, którego nie oddałem (ze względu na znaczący spadek wydajności w przypadku jednego wątku, przyczyna nie do końca jasna), lokalnych map nie ma, jest jedna wspólna, ale żeby nie blokować dostępu do stron za pomocą mutexów, użyłem atomowego += dla double, opartego na CAS idiom.

Poniżej przedstawiam wyniki dla obu wersji. Wyniki dla wersji są uporządkowane następująco:

- 1) czasy Release w runtest.sh na students
- 2) pp. 1 ale lokalnie
- 3) FlameGraph dla performanceTest
- 4) FlameGraph'y dla e2eTest; kolejno dla SingleThread, 1 wątku, 2 wątków, 3, 4 oraz 8.

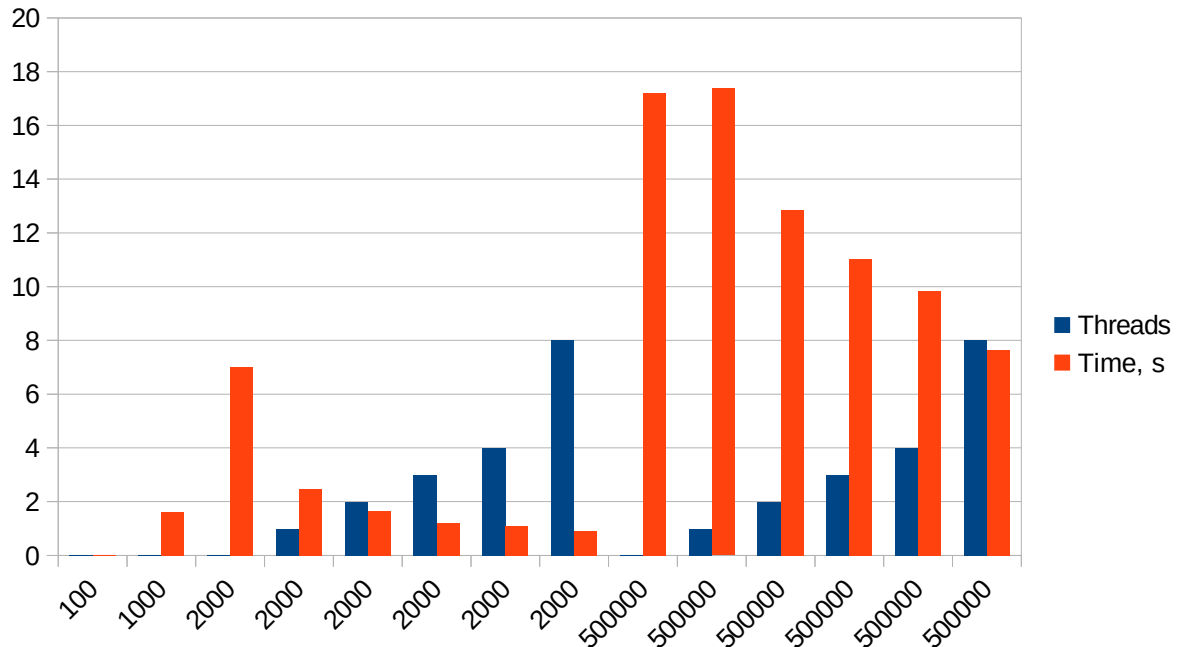
Na każdym z wykresów widać odwrotną proporcjonalność pomiędzy liczbą wątków a czasem wykonania programu, czasami ze wzrostem czasu dla wersji z jednym wątkiem w stosunku do wersji sekwencyjnej. Z coraz większą liczbą wątków przyspieszenie maleje.

Oddana wersja:

- 1) students

PageRank Performance Test [100 nodes, SingleThreadedPageRankComputer] took: 0.00685507s
PageRank Performance Test [1000 nodes, SingleThreadedPageRankComputer] took: 1.60452s
PageRank Performance Test [2000 nodes, SingleThreadedPageRankComputer] took: 7.01584s
PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[1]] took: 2.4522s
PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[2]] took: 1.66061s
PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[3]] took: 1.1974s
PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[4]] took: 1.09877s
PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[8]] took: 0.911969s
PageRank Performance Test [500000 nodes, SingleThreadedPageRankComputer] took: 17.2096s
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[1]] took: 17.3721s
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[2]] took: 12.8656s
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[3]] took: 11.0224s

PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[4]] took: 9.82698s
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[8]] took: 7.62391s



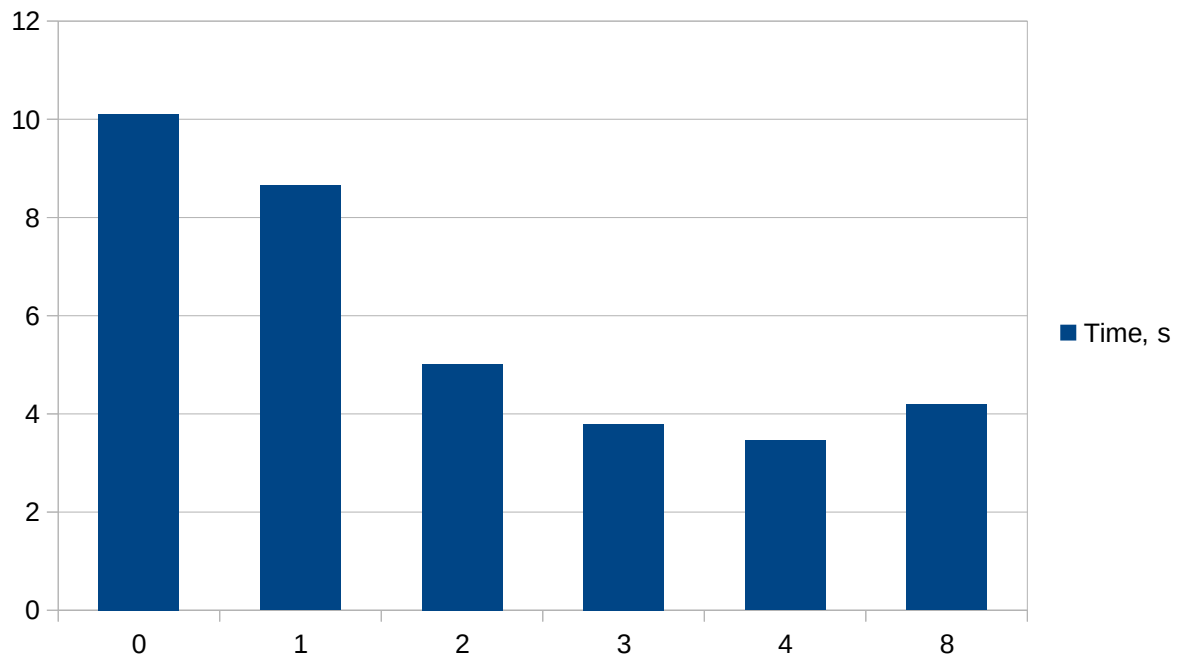
0 w Threads oznacza wersję sekwencyjną.

Nodes	Threads	Time, s
100	0	0.00685507
1000	0	1.60452
2000	0	7.01584
2000	1	2.4522
2000	2	1.66061
2000	3	1.1974
2000	4	1.09877
2000	8	0.911969
500000	0	17.2096
500000	1	17.3721
500000	2	12.8656
500000	3	11.0224
500000	4	9.82698
500000	8	7.62391

E2E Test [SingleThreadedPageRankComputer] took: 10.1075s
E2E Test [MultiThreadedPageRankComputer[1]] took: 8.64788s
E2E Test [MultiThreadedPageRankComputer[2]] took: 5.01734s

E2E Test [MultiThreadedPageRankComputer[3]] took: 3.79164s

E2E Test [MultiThreadedPageRankComputer[4]] took: 3.45243s



E2E Test [MultiThreadedPageRankComputer[8]] took: 4.19304s

Threads	Time, s
0	10.1075
1	8.64788
2	5.01734
3	3.79164
4	3.45243
8	4.19304

lokalnie

PageRank Performance Test [100 nodes, SingleThreadedPageRankComputer] took: 0.00582662s

PageRank Performance Test [1000 nodes, SingleThreadedPageRankComputer] took: 1.21897s

PageRank Performance Test [2000 nodes, SingleThreadedPageRankComputer] took: 4.65461s

PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[1]] took: 1.98385s

PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[2]] took: 1.36723s

PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[3]] took: 1.12946s

PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[4]] took: 1.03533s

PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[8]] took: 0.89976s

PageRank Performance Test [500000 nodes, SingleThreadedPageRankComputer] took: 9.65187s

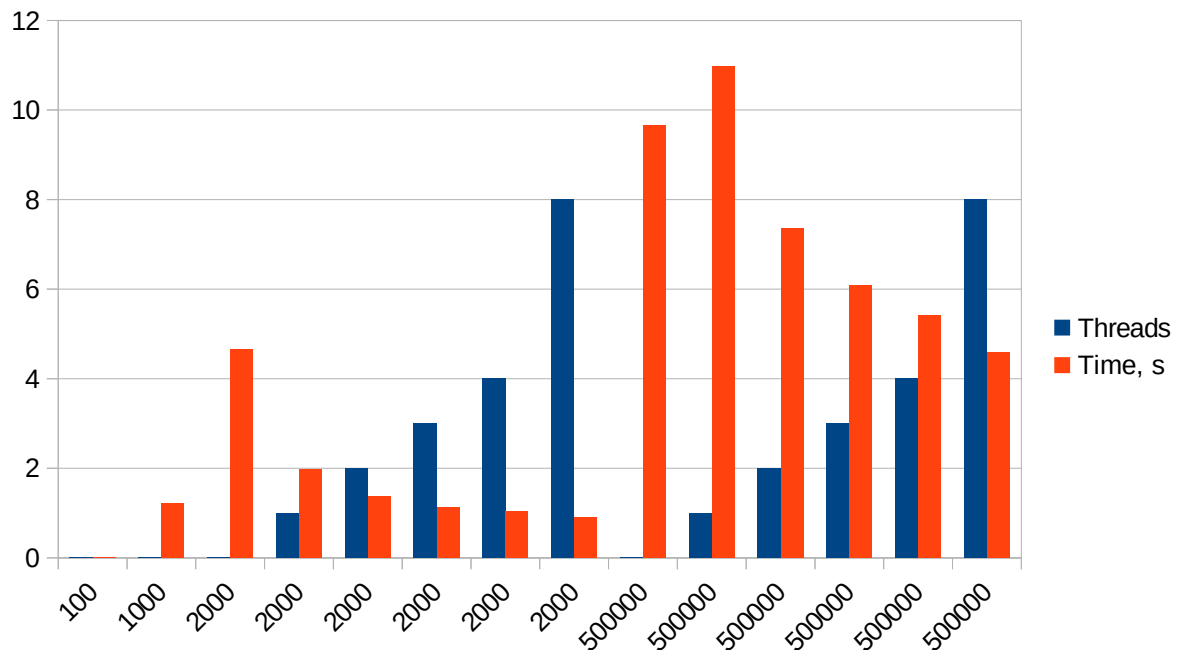
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[1]] took: 10.966s

PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[2]] took:
7.35745s

PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[3]] took:
6.09262s

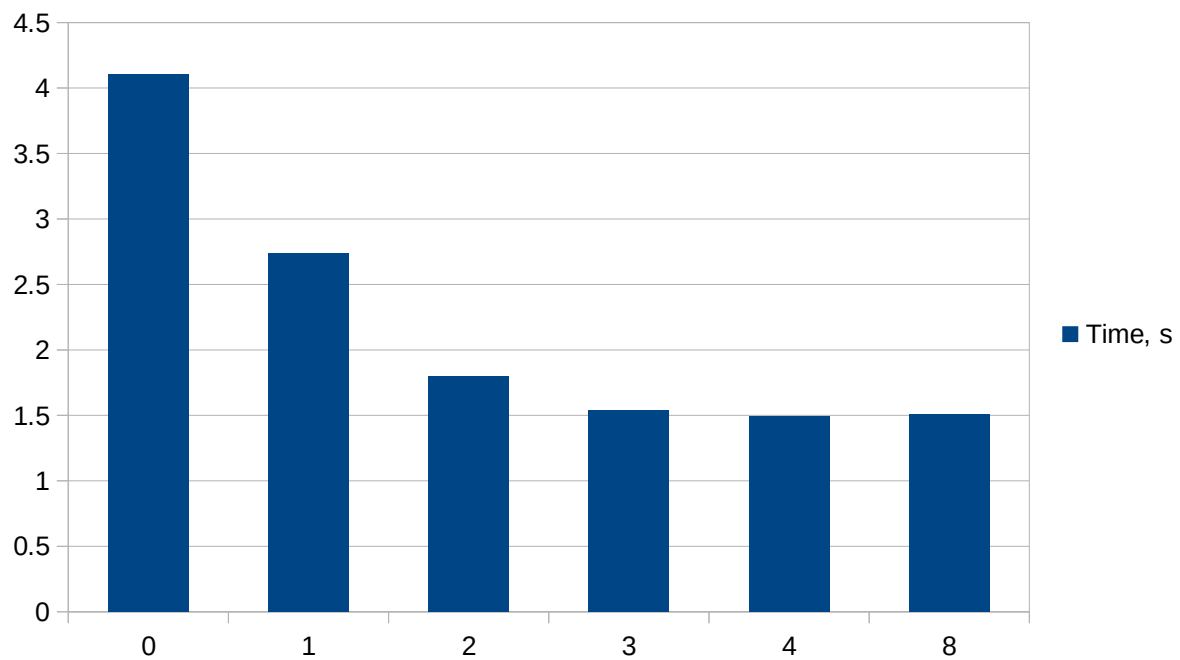
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[4]] took:
5.41386s

PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[8]] took: 4.58072s



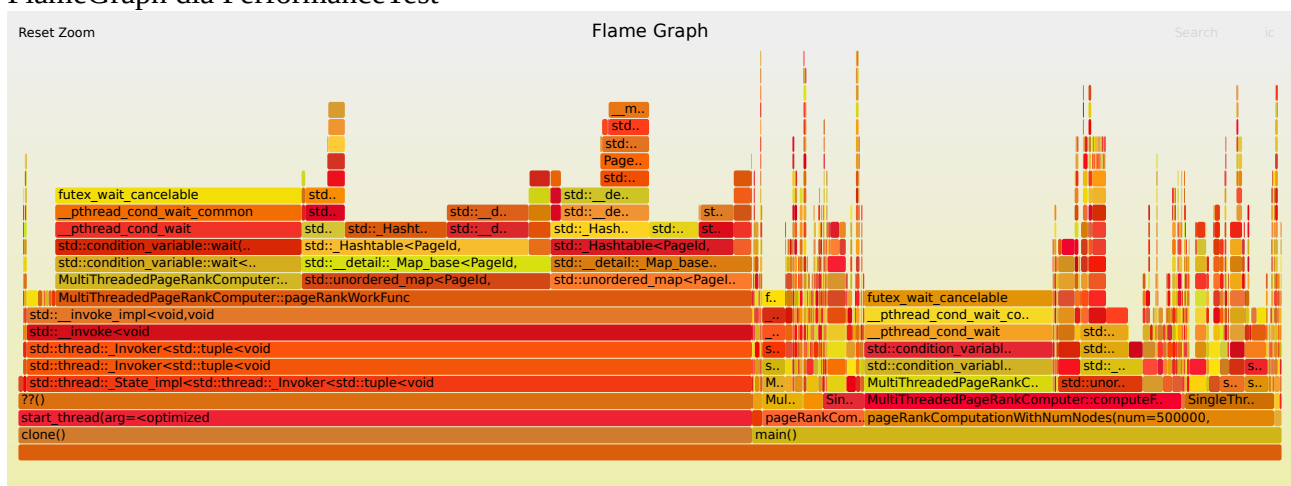
Nodes	Threads	Time, s
100	0	0.00582662
1000	0	1.21897
2000	0	4.65461
2000	1	1.98385
2000	2	1.36723
2000	3	1.12946
2000	4	1.03533
2000	8	0.89976
500000	0	9.65187
500000	1	10.966
500000	2	7.35745
500000	3	6.09262
500000	4	5.41386
500000	8	4.58072

E2E Test [SingleThreadedPageRankComputer] took: 4.10849s
E2E Test [MultiThreadedPageRankComputer[1]] took: 2.73542s
E2E Test [MultiThreadedPageRankComputer[2]] took: 1.79831s
E2E Test [MultiThreadedPageRankComputer[3]] took: 1.54077s
E2E Test [MultiThreadedPageRankComputer[4]] took: 1.49239s
E2E Test [MultiThreadedPageRankComputer[8]] took: 1.50502s



Threads	Time, s
0	4.10849
1	2.73542
2	1.79831
3	1.54077
4	1.49239
8	1.50502

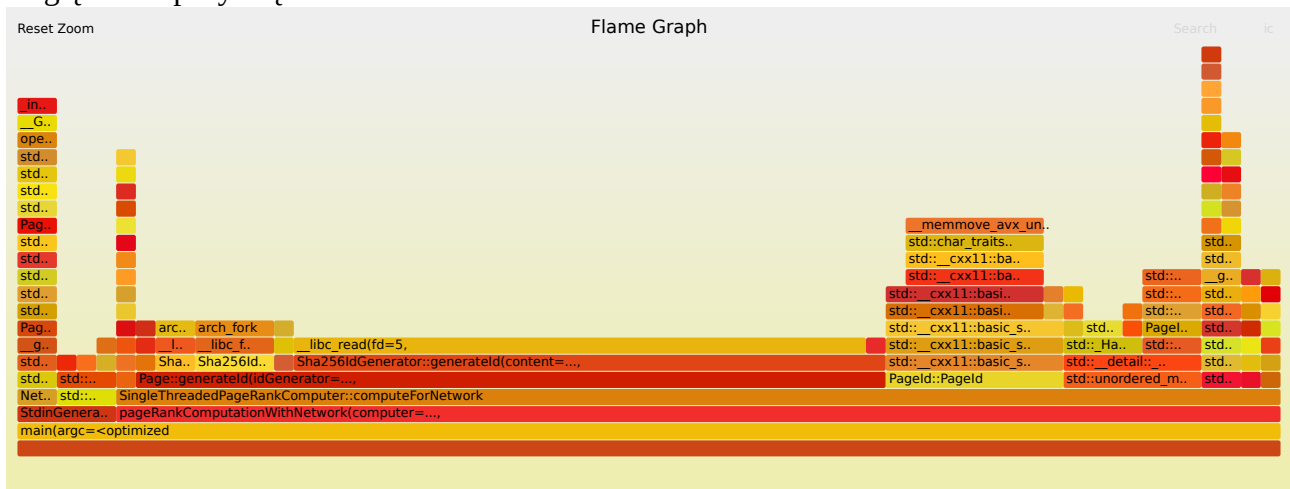
FlameGraph dla PerformanceTest



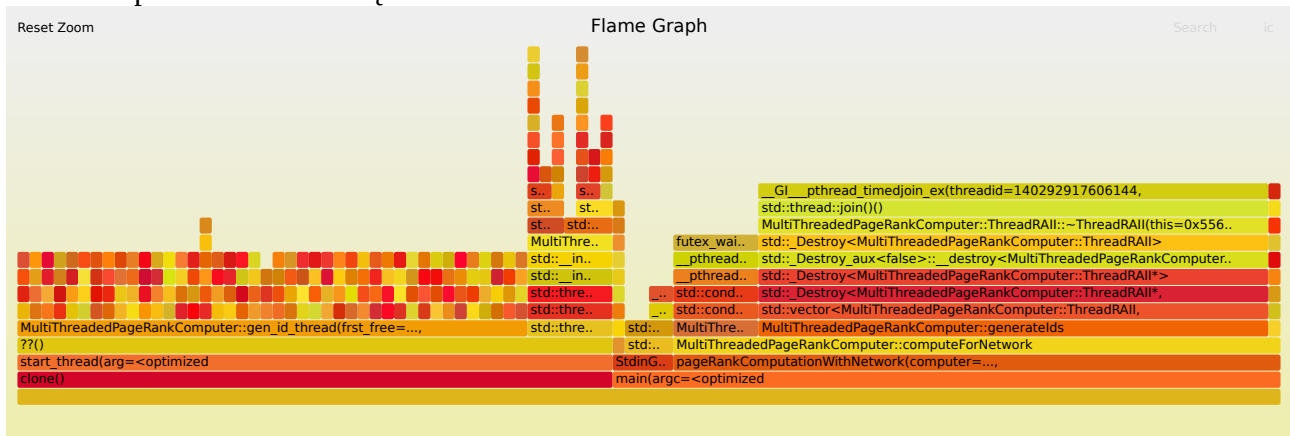
Jak i należało oczekiwać, większość czasu spędza się w wątkach pomocniczych. W wątku pomocniczym ok. 2/3 czasu się spędza na operacja kontenerowych (na unordered map), resztę czekając na bariere. W wątku głównym, najwięcej czasu się czeka na robotę wykonaną przez wątki pomocnicze, inny znaczący kawałek pracy to operacje kontenerowe na unordered map, kombinujące częściowe wyniki wątków pomocniczych.

FlameGraph dla e2e dla Single

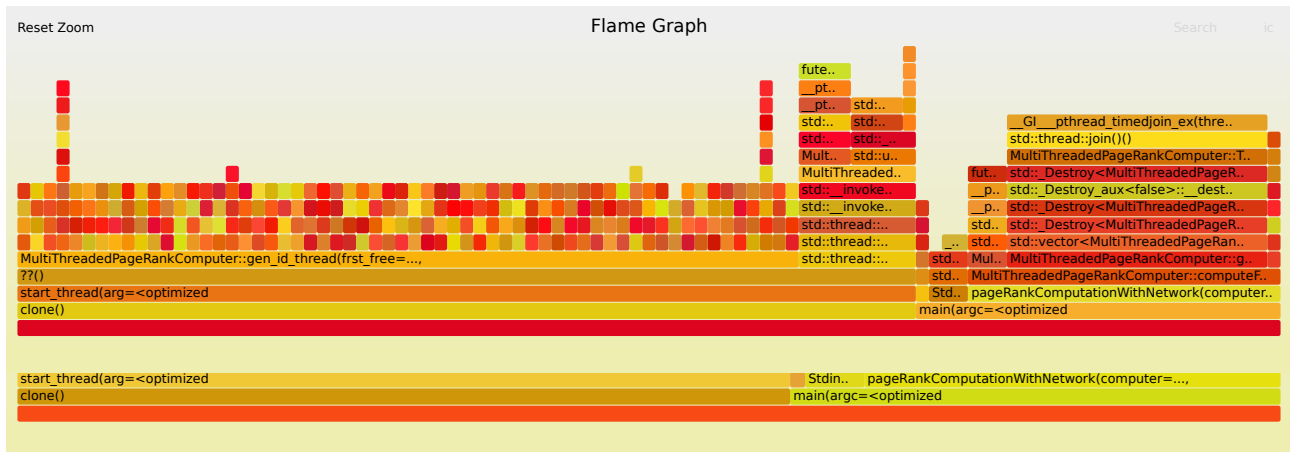
Najwięcej czasu się spędza podczas generowania sha256, czego i należało się spodziewać ze względu na specyfikę testu



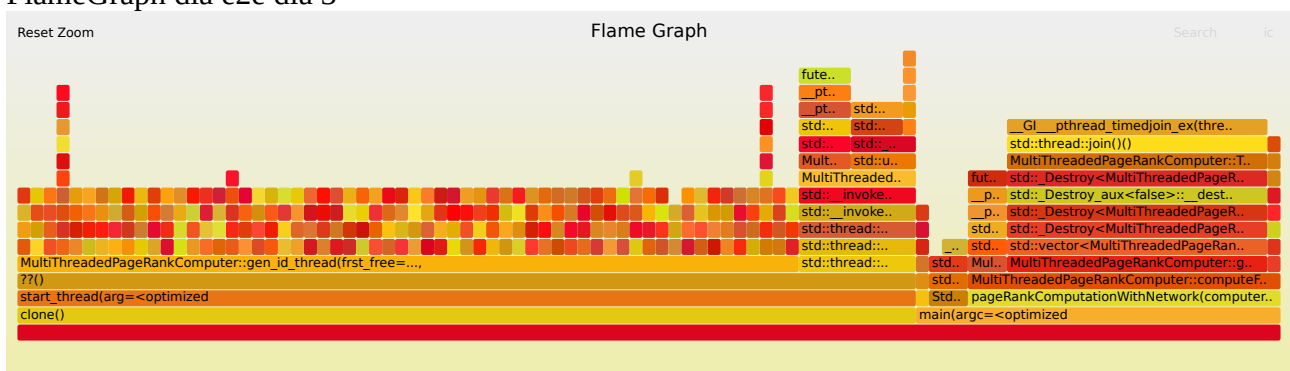
FlameGraph dla e2e dla 1 wątku



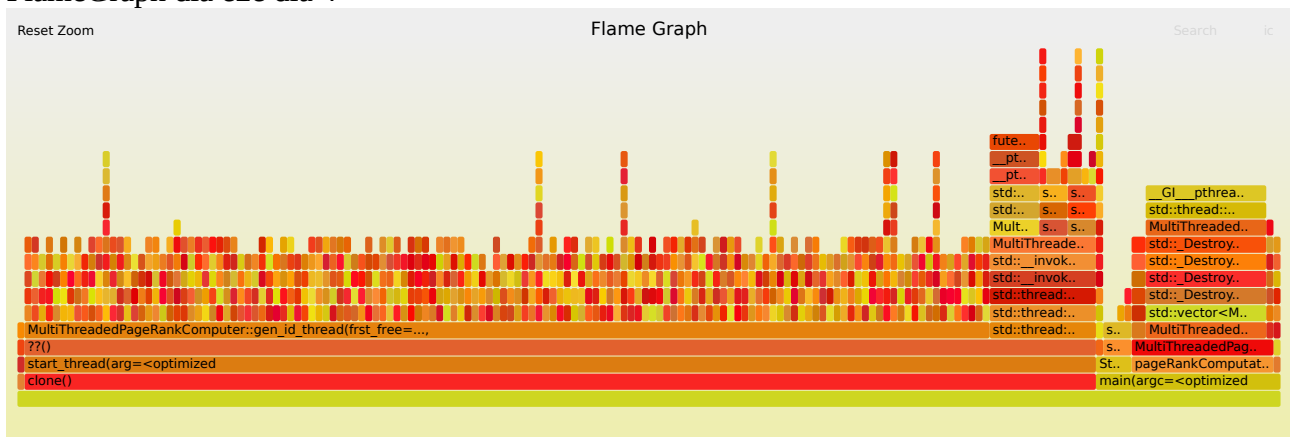
FlameGraph dla e2e dla 2



FlameGraph dla e2e dla 3



FlameGraph dla e2e dla 4



Wersja CAS:

1) students

PageRank Performance Test [100 nodes, SingleThreadedPageRankComputer] took: 0.00994304s
PageRank Performance Test [1000 nodes, SingleThreadedPageRankComputer] took: 1.50444s
PageRank Performance Test [2000 nodes, SingleThreadedPageRankComputer] took: 7.31623s
PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[1]] took: 2.91681s
PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[2]] took: 2.7885s
PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[3]] took: 1.81879s
PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[4]] took: 1.83302s
PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[8]] took: 1.53468s
PageRank Performance Test [500000 nodes, SingleThreadedPageRankComputer] took: 15.0013s

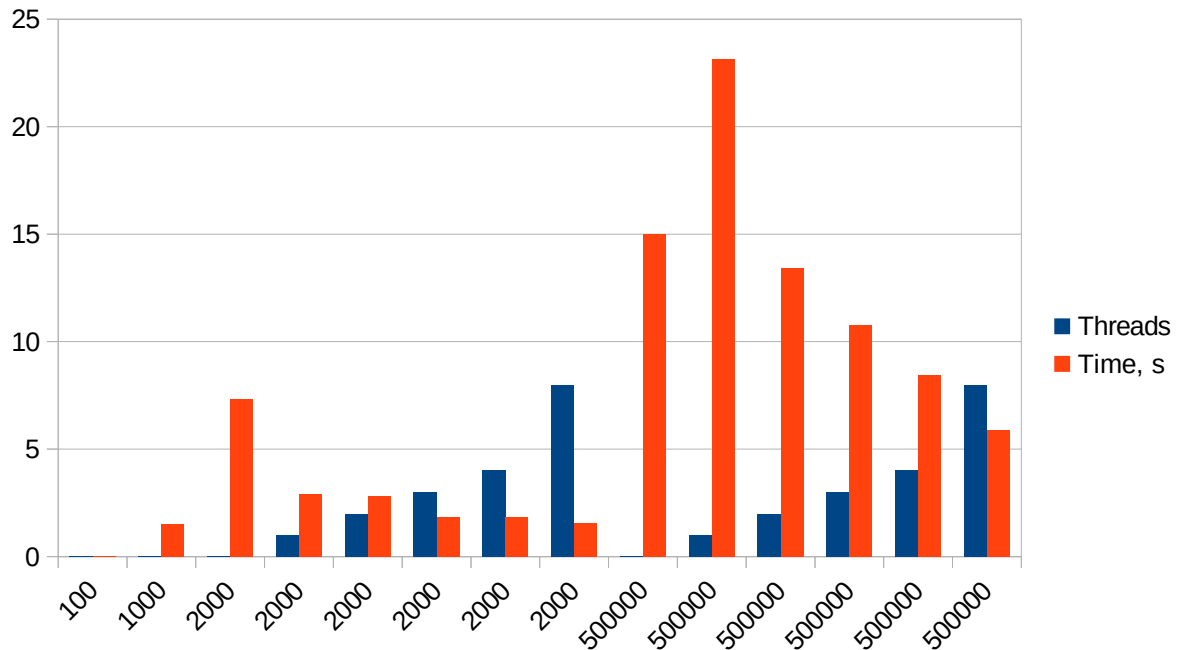
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[1]] took: 23.1311s

PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[2]] took: 13.4159s

PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[3]] took: 10.7522s

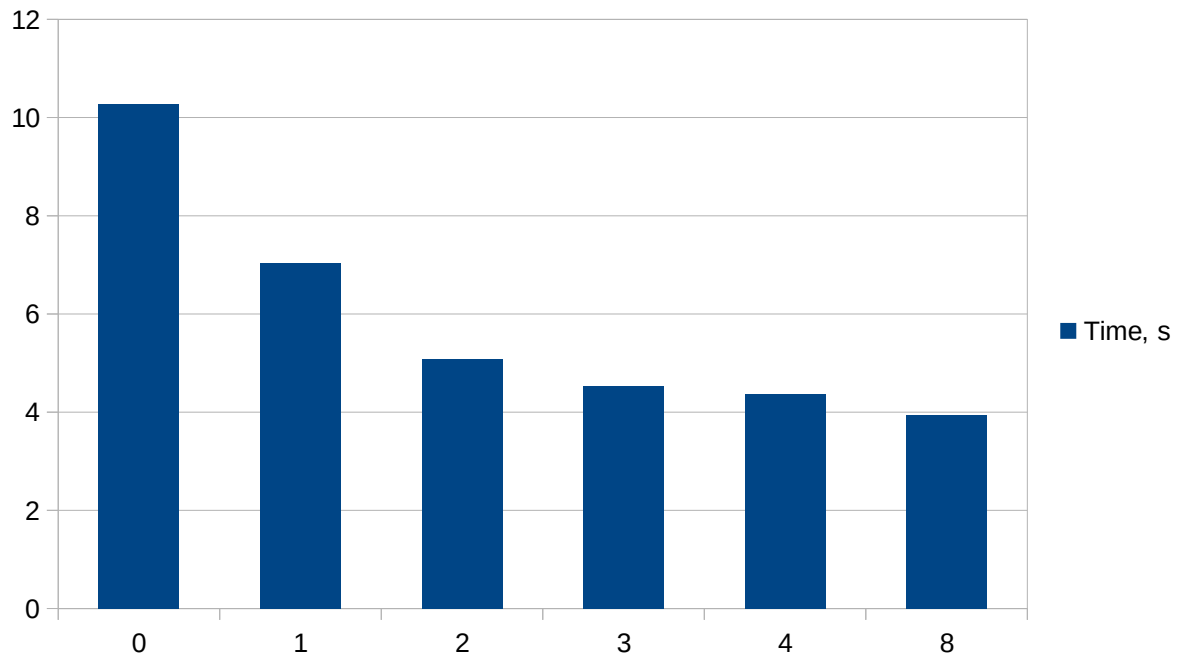
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[4]] took: 8.4244s

PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[8]] took: 5.89626s



Nodes	Threads	Time, s
100	0	0.00994304
1000	0	1.50444
2000	0	7.31623
2000	1	2.91681
2000	2	2.7885
2000	3	1.81879
2000	4	1.83302
2000	8	1.53468
500000	0	15.0013
500000	1	23.1311
500000	2	13.4159
500000	3	10.7522
500000	4	8.4244
500000	8	5.89626

E2E Test [SingleThreadedPageRankComputer] took: 10.2669s
 E2E Test [MultiThreadedPageRankComputer[1]] took: 7.02786s
 E2E Test [MultiThreadedPageRankComputer[2]] took: 5.08296s
 E2E Test [MultiThreadedPageRankComputer[3]] took: 4.538s
 E2E Test [MultiThreadedPageRankComputer[4]] took: 4.37493s
 E2E Test [MultiThreadedPageRankComputer[8]] took: 3.93266s

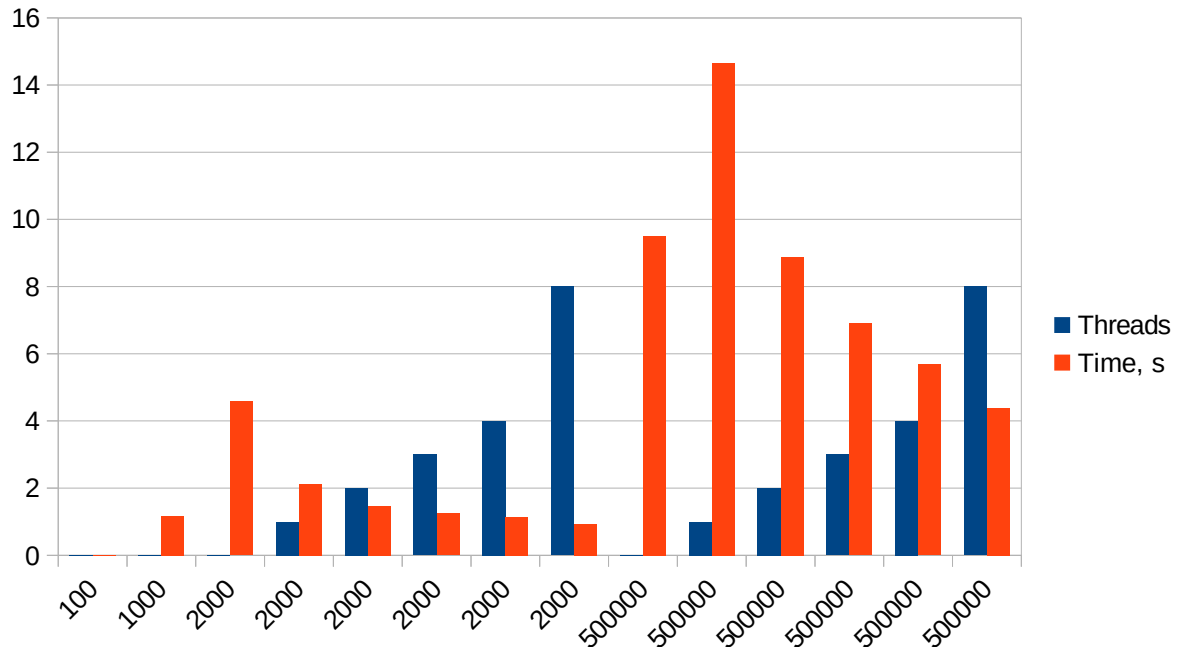


Threads	Time, s
0	10.2669
1	7.02786
2	5.08296
3	4.538
4	4.37493
8	3.93266

lokalnie

PageRank Performance Test [100 nodes, SingleThreadedPageRankComputer] took: 0.00611893s
 PageRank Performance Test [1000 nodes, SingleThreadedPageRankComputer] took: 1.18077s
 PageRank Performance Test [2000 nodes, SingleThreadedPageRankComputer] took: 4.58011s
 PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[1]] took: 2.11881s
 PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[2]] took: 1.46364s
 PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[3]] took: 1.24728s
 PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[4]] took: 1.13407s
 PageRank Performance Test [2000 nodes, MultiThreadedPageRankComputer[8]] took: 0.937209s
 PageRank Performance Test [500000 nodes, SingleThreadedPageRankComputer] took: 9.49957s
 PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[1]] took: 14.6579s
 PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[2]] took: 8.89096s

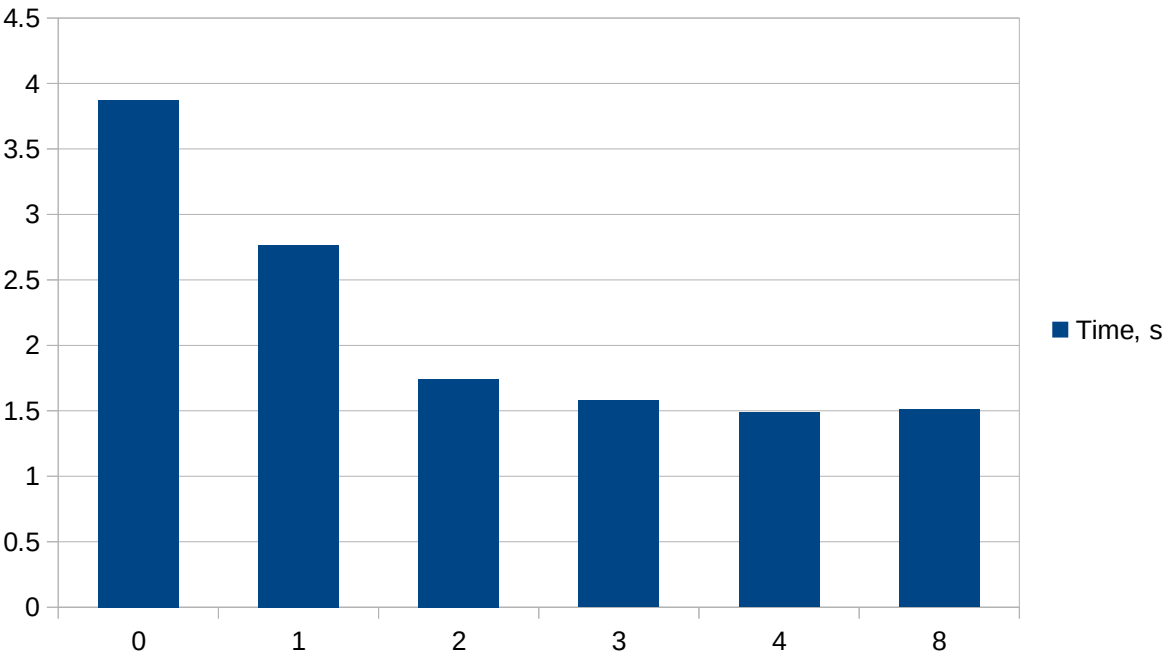
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[3]] took: 6.9005s
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[4]] took: 5.68217s
PageRank Performance Test [500000 nodes, MultiThreadedPageRankComputer[8]] took: 4.38416s



Nodes	Threads	Time, s
100	0	0.00611893
1000	0	1.18077
2000	0	4.58011
2000	1	2.11881
2000	2	1.46364
2000	3	1.24728
2000	4	1.13407
2000	8	0.937209
500000	0	9.49957
500000	1	14.6579
500000	2	8.89096
500000	3	6.9005
500000	4	5.68217
500000	8	4.38416

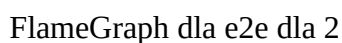
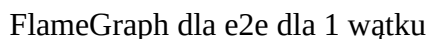
E2E Test [SingleThreadedPageRankComputer] took: 3.87408s
E2E Test [MultiThreadedPageRankComputer[1]] took: 2.76521s
E2E Test [MultiThreadedPageRankComputer[2]] took: 1.74547s
E2E Test [MultiThreadedPageRankComputer[3]] took: 1.57828s

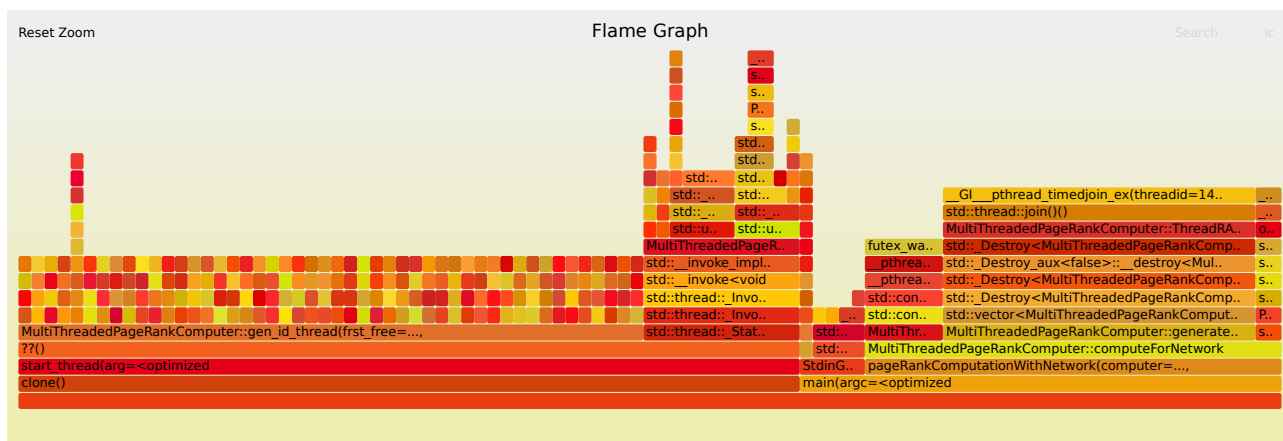
E2E Test [MultiThreadedPageRankComputer[4]] took: 1.4871s
E2E Test [MultiThreadedPageRankComputer[8]] took: 1.51143s



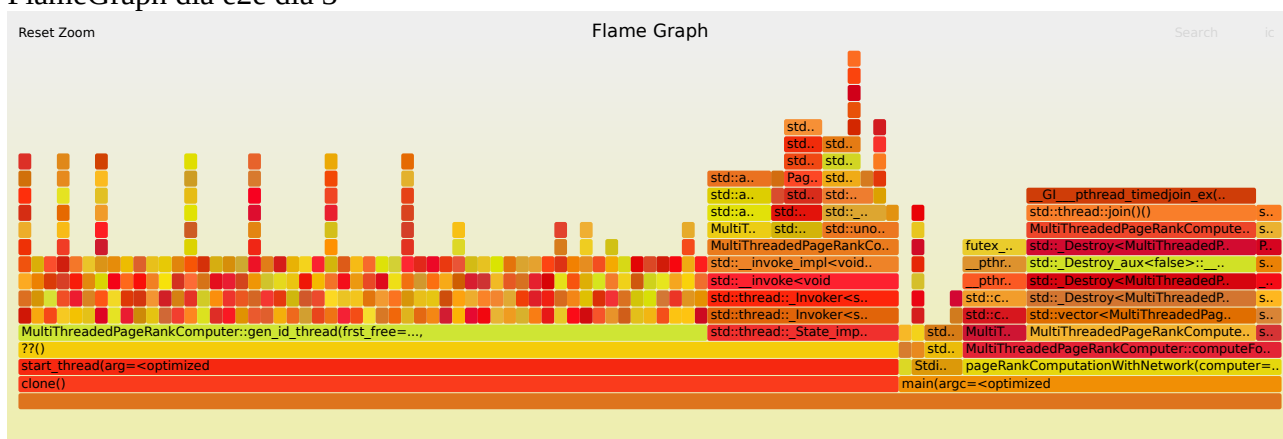
Threads	Time, s
0	3.87408
1	2.76521
2	1.74547
3	1.57828
4	1.4871
8	1.51143

Różnica pomiędzy tą wersją a wysłaną jest taka, że w funkcji roboczej wątków pomocniczych większość czasu się spędza na atomowej operacji inkrementacji. Jest to uzasadnialne tym, że ta operacja nie jest zaimplementowana wybitnie wydajnie, ponieważ CAS idiom jest mniej więcej jak Spin Lock.

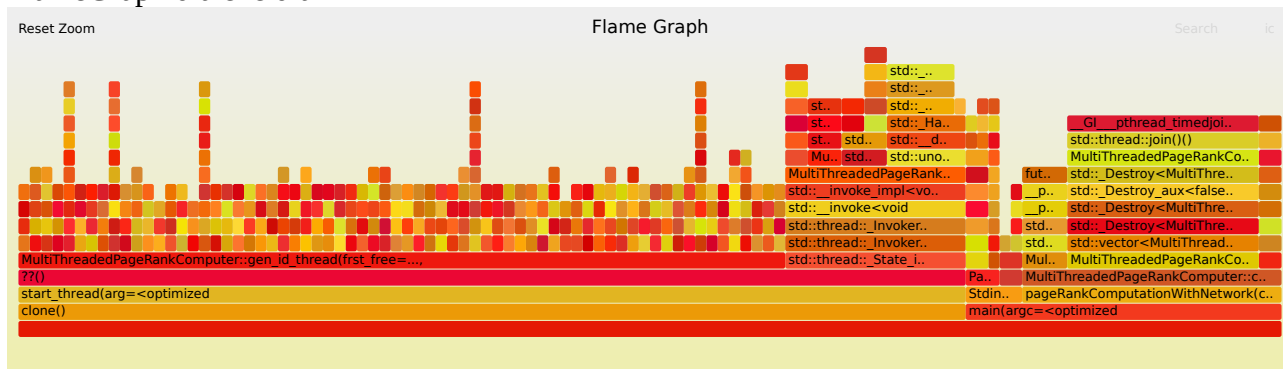




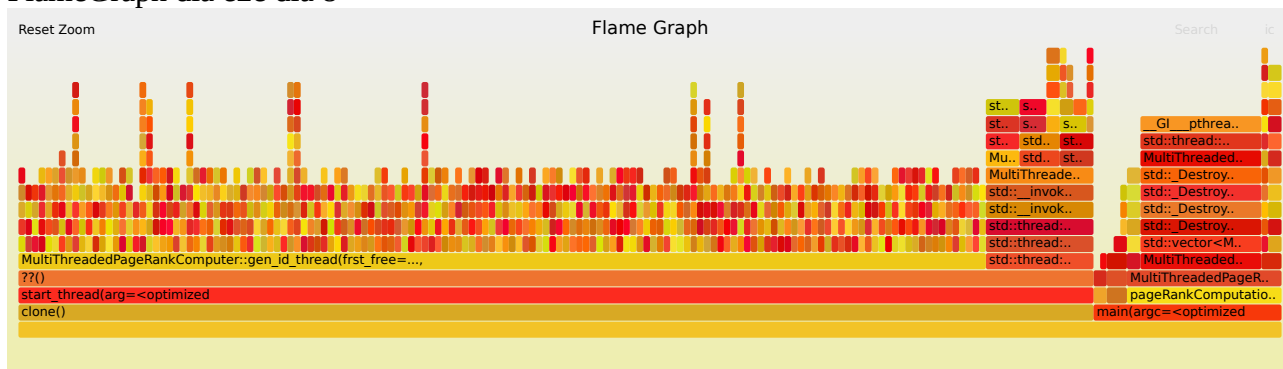
FlameGraph dla e2e dla 3



FlameGraph dla e2e dla 4



FlameGraph dla e2e dla 8



Wykresy dla e2e są bardzo podobne dla wersji CAS jak i dla wersji wysłanej. Cechuje je duży czas spędzony na generowanie id, jego udział rośnie przy zwiększaniu liczby dostępnych wątków a udział przeznaczony na tworzenie i niszczenie wątków maleje.